

1 Considerations of different communication protocols

1.1 Shared bus communication

For the following section the word "port" refers to a connector of the plugin board, not to a controller pin.

Both I2C and SPI, that should theoretically suit this project because of their shared bus architecture, have fundamental issues when trying to incorporate them.

1.1.1 I²C

I2C is a master-slave protocol and provides no support for slaves to initiate communication with the master. This would be required when a new board is connected. This would then mean that all devices would have to be masters or the base board would need to periodically query for new devices. But I2C does not support master to master communication either. The periodical query could be done using a "General Call" where all new devices would respond.

I2C also does not support hot-plugging devices and thus would require additional circuitry.

It also has a non-dynamic addressing scheme which would lead to race conditions and communication collisions. This could be mitigated by using the same address for all addresses and then checking on a higher layer whether the packet is actually meant for the given device.

I2C also has high baseline power consumption because of the pull-up resistors.

1.1.2 SPI

SPI also is a master-slave protocol and thus would make registration with the base board complicated. Another problem is that SPI all variations of the bus topology do not work for hot plugging. - Multidrop configurations would require the blocks to do the routing themselves by setting their neighbours CS. The master would select one of its neighbouring blocks and tell it to set the CS lines of one of their neighbouring blocks. The master can now address this block and tell it to set the CS line of one of that blocks neighbours. This would have to be done recursively for each communication between base and target block. - Daisy chain configurations cannot ever work since the chain of MISO/MOSI chain cannot be broken. One would have to bridge across ports that are not connected, but still enable them to connect using some other way.

1.1.3 Other Options

I2C, SPI and UART are the most widespread communication options on microcontrollers. Since I2C and SPI are both out, this leaves only peer-to-peer communication using UART.

Here I could only find two distinct variants for communication: - Parallelism - All ports are connected to separate hardware UART's. - This is the easiest way but has significantly higher hardware requirements which are not required as the boards will be in sleep mode for most of the time. - Time sliced - Only one hardware UART needed for an arbitrary amount of ports. - Every block has only one port at a time whose communication lines are actually connected to the hardware UART. Two devices need to synchronize their active ports to start communication. This active port can be switched to another port when communication is finished. - This variant is also compatible with the other case if the device with the parallel ports stores messages until they are acknowledged. - The non-active ports pull the sending line to high as this is the default state for UART and when they want to communicate they pull the line low which corresponds to the UART start bit. - Slightly higher hardware requirements: 1 Demultiplexer in general and 1 resistors, 1 window comparator, 2 transistors, 1 pin more per port.

The first variant could detect whether a block is connected to one of its port using periodic discovery requests. But it could also use a window comparator for detecting this which would reduce the computational load of the microcontroller and make it asynchronous.

2 Communication Protocol

Communication occurs over a multiplexed UART interface.

In the following, each side of a block that can be connected to, will be called "port".

The amount of ports of a block will be denoted as P .

The communication can be extended to arbitrarily many ports.

2.1 Pinout

The following are the required pins on the microcontroller side:

- UART RX and TX
- $\lceil ld(P) \rceil$ Pins for selecting which port is active
- P Pins for Interrupts

The following are the required pins between the blocks:

- VCC
- GND
- UART TX
- UART RX

2.2 Communication Process

For the following section the TX and RX of the microcontroller and block connection will be qualified by "uC" for the microcontroller and "bc" for the block connection respectively.

At any given time one of the ports will be selected (excluding during port selection transitions).

The selected port will have its bc_TX connected to uC_RX and bc_RX to uC_TX thus enabling full-duplex communication.

For the ports that are not selected the following configuration is used:

The bc_RX line will be pulled to high to indicate to the other block that a block is connected.

At the same time a floating detector is used on the bc_TX line to check if a block is connected.

The interrupt pin will be configured with a falling edge interrupt.

If you have data to send to the another block you set the given block to the active port and send a connection request frame.

Then you set a timer for 1ms during which you wait for a UART response frame.

If the other block does not respond before the timer runs out, you can move on to a different block or retry the current block.

(This should probably done with SYN -> ACK -> SYN-ACK where if the timer runs out you restart with SYN)

Here we have to make a case distinction:

- If the other block has this port as the selected port:
It will immediately get the frame and communication can start immediately.
- If the other block does not have this port as the selected port its RX will be connected to the interrupt pin.
The UART start bit of the frame will pull the pin low and trigger a falling edge interrupt on the other block.
If the other block is currently idle it will then change its selected port to the port where the frame was sent.
If the other block is currently communicating with another block, it will finish this communication with this block and then reinitiate the communication with the block that sent the request.
- If the block is currently changing the selected port from this block to another block, it will miss the communication request.
This will be caught by the timer reinitiating the request after some time.

2.3 Circuit Design

I did create a circuit design on paper but was not able to convert it into LaTeX Code yet.